Principles & Practices — How I Build My Project with AI

E. Sai Shervan (CS24B073) · CS5013 Programming with AI · Mid-evaluation, October 2026
Project: *Interactive Multistatic Radar Simulator for 3D Drone Localization* · Stakeholder: Dr. Giridhar K (EE) · Repo: `github.com/shervan0714/MultiStaticRadarSimulator`

My project is a Java/JavaFX simulator: you place a transmitter and several receivers, move a drone along a trajectory, simulate the radar measurements, estimate the drone's 3D position, and compare it with the true position. I use an AI coding assistant (Claude Code, running inside my repository) for much of the implementation. My rule is that the AI is a fast collaborator, not an authority: **I decide what gets built, every change has to pass a check I can run myself, and I must be able to explain every line**. This document describes how I apply that rule.

1. Prompt engineering

Looking back at my prompt history (kept in `prompts/PROMPT-LOG.md`), these are the principles that worked, and the ones I learned from my weaker prompts.

- **Understanding before code.** My most useful prompt asked the assistant to read my proposal, design doc and code and tell me *how far the project really was*, before changing anything. It built and ran everything and found that localization was 80–115 m off on noiseless data, while all 76 tests still passed — because the tests only checked that the error was a finite number. Asking for code first would have built more on a broken base.
- **Plan, then approve.** I ask for a plan split into commit-sized steps and approve it (e.g. "say me what is there to do then i will say to do or not"). This keeps the scope mine and forces me to read each proposed change.
- **One concern per prompt.** Each request maps to one commit (build fix, solver, tests, ID bug…). Small steps are easier to verify and to undo.
- **State the check, not just the task.** "Tests must pass", "noiseless cases within 1 mm", "show me the app running". Without a check I can't tell good output from plausible-looking output.
- **Be specific about symptoms.** One weak prompt — "use shervan0714 github not karthiks" — named a symptom without saying where I saw it, so the assistant had to investigate everything. (It still found a real issue: my commits were not linked to my GitHub account because of the commit email.) Now I say *what* I saw, *where*, and what I expect.
- **Ask "why".** When I don't understand a result (e.g. why there were empty folders, why the solver needs several starting points), I ask before accepting it. I will be asked to explain it in the viva.

The domain mathematics is **not** taken from the AI as authority: the stakeholder's formulation will decide the final model, and the current baseline is checked against known ground truth. Sample prompts, with how I would improve them, are in the appendix.

2. Debugging

I follow the same loop for every bug: **reproduce → measure → isolate → form a hypothesis → minimal fix → add a regression test → record it**. Four real cases from this project:

Symptom	How it was isolated	Root cause → fix	Regression guard
Estimates ~100 m off on *noiseless* data, yet all tests green	Ran the CLI for 3/4/5 receivers to measure the error; read the tests — they asserted only `isFinite(error)`. Error should be ~0 with perfect data, so the solver itself was wrong.	Power-weighted guess + finite-difference gradient descent with z forced to 500 m. Replaced with least squares on bistatic-range ellipsoids (Levenberg–Marquardt, analytic Jacobian).	Reference scenarios must match truth within **1 mm**
`gradlew test` crashed: "Unsupported class file major version 70"	Version 70 = Java 26 class files; the wrapper pinned Gradle 8.10, which cannot run on Java 26. Confirmed by running the same build with a cached Gradle 9.3.	Upgraded the wrapper to 9.3.0.	README build steps; wrapper committed
Last sample sometimes missing from a run	Time was accumulated as `t += dt`; with `dt = 0.1`, 0.1+0.1+0.1 = 0.30000000000000004 > 0.3, so the loop stopped early.	Compute time from the step index: `t = i · dt`.	Test: duration 0.3, step 0.1 → 4 records

New test "target on the ground is unstable" failed	Wrote a small probe printing estimate, residual and condition number for heights 0–100 m. The estimate *was* correct; only the condition number exploded (1.9×10^7 vs 33).	The **test's assumption** was wrong, not the code. Rewrote the test to check the warning signal (large condition number).	Test kept, with the corrected expectation

Two lessons: a test suite that cannot fail gives false confidence, and when a test fails I first decide whether the *code* or the *expectation* is wrong. For the GUI, where tests cannot see the screen, I also check rendered screenshots (that is how an upside-down Y axis in the 3D view was caught).

3. Version control (Git) — for code and prompts

- **Small, purposeful commits** with `type(scope): summary` messages (`fix(localization)`, `test`, `refactor`, `docs`) and a body that says *why* and what was measured. My early message "rewrote localization math..., accuracy is way better now" claimed an improvement that was never measured (the error was still ~100 m); now messages report numbers.
- **Branch, then fast-forward.** Work happens on `mid-demo-fixes`; after the full test suite passes, `main` is fast-forwarded and pushed, so `main` always builds.
- **History is preserved.** When I split the 21-class `model` package into the design doc's modules, files were moved with `git mv` (27 renames), so `git log --follow` still shows each file's history.
- **AI help is visible.** Commits made with the assistant carry a `Co-Authored-By` trailer, so the history is transparent about where AI was involved.
- **Prompts are versioned with the code.** `prompts/PROMPT-LOG.md` records each prompt verbatim with its date, goal, outcome, how it was checked, and the commit hash it produced, plus a rewritten "better version". It is committed alongside the code changes, so prompt → change → verification can be traced.
- **Hygiene.** `.gitignore` keeps build output out; `.gitattributes` fixes line endings for scripts. I also fixed my commit email so commits link to my GitHub account (older commits used an address not on my account).

4. Testing, refactoring and documentation

Testing

- **111 JUnit tests** (up from 76), with at least one test class per design doc module: vectors and 3×3 maths, trajectory, measurement, localization, geometry validation, experiment runner, evaluation, persistence, and the GUI's logic.
- **Ground truth as the oracle.** Because the true drone position is known at every step, noiseless reference scenarios (3, 4 and 5 receivers, wide baseline, close range, elevated transmitter) must reproduce it within 1 mm.
- **Failure cases are tested too.** Too few receivers, collinear sensors, duplicate IDs, physically impossible ranges and badly conditioned geometry must return an explicit status (`UNDER_CONSTRAINED`, `INVALID_INPUT`, ...), never a silent wrong position.
- **Testable design.** GUI logic that can be pure Java is pure Java (`ScenarioConfig`: form → scenario; `SceneTransform`: East/North/Up → JavaFX), so it is unit-tested without a display.

Refactoring

Refactors are behaviour-preserving and guarded by the test suite: the package restructure changed only packages and imports, and all 111 tests passed before and after. Unused code was removed (an unconnected `ParameterPanel`, a frame-by-frame loop that bypassed the experiment pipeline), and the coupled `RadarPhysics` class from the early prototype was split into replaceable `MeasurementModel` / `LocalizationModel` interfaces, as planned in the design doc.

Documentation

- `README.md` — build/run steps, GUI and CLI usage, coordinate convention, architecture, verification, contact.
- `docs/PROJECT-STRUCTURE.md` — what every file is for, and "where to change what".
- `docs/MID-DEMO.md` — demo script and an **honest list of 12 known gaps** (e.g. noiseless measurements, placeholder power formula, stakeholder formulation not yet received).
- `docs/WEEKLY-LOG.md` — weekly contributions, doubling as the change log; Javadoc records modelling assumptions (e.g. the "target above the sensors" choice for coplanar layouts).

Appendix — Sample prompts

Prompts are quoted **exactly as I wrote them** (from `prompts/PROMPT-LOG.md`). Under each is how I would write it now, applying the principles in Section 1.

A1. Understand the project before changing it (7 Oct)

> see my design doc and proposal and understand my project first and then see to how much extent my project is done and then plan what to do divide them into commits

Result: status report against the design doc; found 80–115 m error hidden by tests that could not fail; 21-commit plan.

> Read cs5103.pdf (proposal) and desgin_doc.pdf (Phase 3 design) and the code in MultiStaticRadarSimulator/. For each module in design doc §1.3, tell me whether it is done, partial or missing, with evidence (file, test or command output). Build it and run the tests and the CLI — don't trust the README. Then propose small commits, most important first, saying how each will be verified. Don't modify any files yet.

A2. Implement with checks and a change record (7 Oct)

> yes start and write the changes if there is file like track.md or somthing whenre previous changes are noted

Result: 4 commits (build fix, new solver, 1 mm tests, receiver-ID fix); tests 76 → 90; error ~100 m → 0.0000 m.

> Do commits 1–4 from the plan, one at a time. After each, run ./gradlew test and the CLI and show me the numbers, then commit with a message that says what changed and why, and add a bullet to Week 5 of WEEKLY-LOG.md. If a test fails, stop and explain whether the code or the test is wrong before fixing either.

A3. Report a problem precisely (7 Oct)

> i want you to use shervan0714 github not karthiks

Result: push account and remote were already correct; found that commits were not linked to my account (wrong commit email) and fixed it for new commits.

> On GitHub I see "karthiks" next to my commits in the commit list (screenshot attached). Check which account the pushes and commit authors use, and make sure everything in this repo is attributed to my GitHub account shervan0714.

A4. Plan first, I decide (8 Oct)

> (pasted the TA's mid-evaluation announcement) do necessary work first say me what is there to do then i will say to do or not

Result: checklist of deliverables vs. what already exists — e.g. no prompts were tracked in git, which led to the prompt log.

> Here are the mid-evaluation guidelines. Map each required item to what already exists in my repo (with file paths) and what is missing. Estimate the effort for each missing item and suggest an order given the Sunday deadline. Don't create or change anything until I choose.

A5. Safe refactoring (11 Oct)

> format the folder and file strucutre properly if it isn not in a clean way , ,create a file and say what each file is for

Result: model package split into the design doc's modules with git mv (27 renames, history kept); all 111 tests still pass; docs/PROJECT-STRUCTURE.md written.

> Reorganise the source packages to match the modules in my design doc §1.3. Use git mv so history is kept, change nothing except packages and imports, and run all tests before committing. Then write docs/PROJECT-STRUCTURE.md explaining every file, so I can find my way around during the live code task.

A6. Asking to understand, not just to fix (template I plan to use while preparing for the viva)

> Explain BaselineLocalizationModel to me as if I'm presenting it: what equation each receiver gives, why Levenberg–Marquardt is used instead of plain gradient descent, why it starts from several points, and what the residual and condition number tell me. Then give me two questions an examiner might ask about it, and let me answer before you check.

Prompt checklist I now follow: (1) give context and point to files; (2) ask for understanding or a plan before code; (3) one concern per prompt; (4) state how the result will be checked; (5) say what must not change; (6) ask "why" until I can explain it; (7) log the prompt with the commit it produced.